

CENG499 Homework 2 Report

Name: Adilhan Çetin

Number: 2580421

April 5, 2026

Question 1

a)

The test performance of the perceptron model for both 10 and 20 epochs is presented below:

Epoch = 10 Results:

- Accuracy: 0.9509
- Precision: 0.9412
- Recall: 0.9917
- F1 Score: 0.9658

Epoch = 20 Results (Best):

- Accuracy: 0.9653
- Precision: 0.9752
- Recall: 0.9752
- F1 Score: 0.9752

As we can see from the metrics, increasing the number of epochs results in a noticeable improvement in our test performance metrics. The accuracy metrics has increased significantly, increasing from 95.1% to 96.5%. More importantly, precision and recall became perfectly balanced at 20 epochs.

When we look back at the model from the first homework(decision tree implementation) evaluated on the same dataset, the performance was definitely lower: Accuracy of 0.9362, Precision of 0.8719, Recall of 0.8860, and an F1 Score of 0.8787. The perceptron outperforms the decision tree here because our one-hot encoding projected the categorical data into a highly linearly separable, sparse space. The perceptron easily finds a linear boundary in this space, whereas the decision tree struggles to make efficient splits across many sparse, binary features.

b)

Increasing the number of epochs from 10 to 20 noticeably improved the model's performance on the test set. Accuracy rose from 95.09% to 96.53%, and the F1 score improved from 96.58% to 97.52%. These additional epochs provided the perceptron more iterations to correct misclassifications on the training data and refine its weights, resulting in a more optimal decision boundary.

However, we can not say such a change is always expected. We can see that some of the metrics on 20 epochs are nearly exactly the same. That means that our learning model has nearly converged. If we increase epochs to more than 20, we might now get a significant improvement. In fact, if we keep increasing epochs, we might even start to see overfitting where the model starts to memorize the training data and performs worse on unseen test data. Therefore, while increasing epochs can improve performance, it is not guaranteed and should be monitored carefully.

c)

My implementation begins by fetching the Car Evaluation dataset from the `ucimlrepo` API. Then I split data to training and test partitions with the given hyperparameters. Since the perceptron algorithm relies on numeric data (rather than just categories compared to decision tree algorithm), I wrote a custom `encode_features` function that performs one-hot encoding. After encoding, I initialize the weight vector and bias to zero. In the training loop, the model iterates through the training data for the specified number of epochs. For each instance, it calculates the input features and weights, plus the bias. If the model makes an incorrect prediction, the weights and bias are updated according to the standard perceptron rule. Finally, I evaluate the model on the test set by manually calculating the true positives, false positives, true negatives, and false negatives to compute the required metrics: accuracy, precision, recall, and F1 score.

d)

There are several modifications that could improve the perceptron's performance. First, instead of initializing the weights and bias to zero, we could use small random initial weights. This helps the model by starting from a more diverse initial state, which can lead to faster convergence since it doesn't start from a completely neutral state.

Second, we could use a decaying learning rate rather than a fixed one. With a large, fixed learning rate, the weights can change quite fast around the optimal solution without settling. If we slowly decrease the learning rate as the epochs progress, the perceptron can make smaller, more careful adjustments and settle cleanly on the best decision boundary.

e)

Results from part a shows that this dataset is highly suitable for the perceptron algorithm. Categories can be easily separated in the one-hot encode algorithm.

Question 2

a)

The test performance for all three Neural Network architectures is reported below:

1 Hidden Layer (16 neurons):

- Accuracy: 0.9214
- Precision: 0.9348
- Recall: 0.9323
- F1 Score: 0.9334

2 Hidden Layers (16-16 neurons):

- Accuracy: 0.9266
- Precision: 0.9390
- Recall: 0.9364
- F1 Score: 0.9376

1 Hidden Layer (32 neurons):

- Accuracy: 0.9229
- Precision: 0.9362
- Recall: 0.9320
- F1 Score: 0.9339

All three models performed very well, giving around 92–93% test accuracy. However, the model with 2 hidden layers (16-16 neurons) performed slightly better on every metric. The reason is that the additional layer allows the model to learn more complex relationships in the data, which can lead to better generalization. The 32-neuron single layer model has more capacity than the 16-neuron model, but it may not be as effective at capturing the underlying structure of the data as the deeper 16-16 model. Overall, the results suggest that increasing depth can be more beneficial than just increasing width for this dataset.

b)

I started by getting the dataset and scaling the numeric features with `StandardScaler`, which is important to prevent training errors in neural networks. I also converted the string labels into integer categories (0-6) using `LabelEncoder`. Then, I split the data into 70% training, 10% development, and 20% test sets and converted them to PyTorch tensors.

The network models inherit from `nn.Module`. They pass inputs through their `nn.Linear` hidden layers, use an `nn.Sigmoid` activation function, and output to 7 classes. In the training loop, I calculate the predictions and loss, compute the gradients with `loss.backward()`, and update the weights using the `Adam` optimizer. Lastly, I evaluate the models on the dev and test sets.

c)

For my hyperparameter search, I focused on the learning rate and the number of epochs. I tried learning rates of 0.1, 0.01, and 0.001, and epoch values of 100, 200, and 500. A learning rate of 0.1 caused the loss to fluctuate too much, while a rate of 0.001 with 100 or 200 epochs caused underfitting, giving only around 85% accuracy on the dev set. I found that a learning rate of 0.01 with 500 epochs was the best setting, resulting in a stable 93.6%-94.0% dev-set accuracy.

For the activation function, I used Sigmoid because it reliably keeps the layer outputs between 0 and 1. For the loss function, I used Cross-Entropy Loss (`nn.CrossEntropyLoss()`) because this is a multi-class classification problem. This loss function perfectly handles categorical integer targets in PyTorch.

d)

When comparing the three models, the minor performance differences show some normal behaviors. First, the 32-neuron single layer model has more capacity than the 16-neuron model, which explains why its test metrics are slightly better. However, the two-layer network (16-16) performed the best overall. The reason behind this is that the additional layer allows the model to learn more complex relationships in the data, which can lead to better generalization. The 32-neuron single layer model has more capacity than the 16-neuron model, but it may not be as effective at capturing the underlying structure of the data as the deeper 16-16 model. Overall, the results suggest that increasing depth can be more beneficial than just increasing width for this dataset.